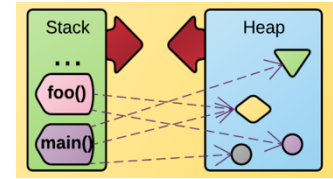


Busca Palavras

Patrícia Dockhorn Costa, pdcosta@inf.ufes.br



Nesta prova, você precisa implementar um programa que verifica se uma determinada palavra existe em uma matriz dinâmica de palavras. A matriz de entrada será dada em um arquivo texto (entrada.txt), como no exemplo a seguir:

```
4 5
Estrutura de Dados Ciencia Computacao
Engenharia Ufes terceiro periodo fatorial
Eletrica Estatistica Matrices Dinamicas exemplos
Material para prova vetores ponteiros
```

A primeira linha contém 2 números (4 e 5) que representam as dimensões da “matriz” de palavras. A palavra “Ciencia”, por exemplo, está na posição [0][3]; “Matrizes” está na posição [2][2]; “Material” está na posição [3][0].

Após carregar essa matriz, seu programa deve imprimir a matriz no terminal e gerar um “prompt” para o usuário com duas opções: (1) a possibilidade de sair do programa; ou (2) digitar uma palavra a ser encontrada na matriz. Por exemplo, o prompt do meu programa ficou assim:

```
Estrutura de Dados Ciencia Computacao
Engenharia Ufes terceiro periodo fatorial
Eletrica Estatistica Matrices Dinamicas exemplos
Material para prova vetores ponteiros

=====
==> Digite a palavra para o caça-palavras ou 0, caso queira sair:
Estrutura
Palavra encontrada na posicao [0][0] :)!

==> Digite a palavra para o caça-palavras ou 0, caso queira sair:
Ciencia
Palavra encontrada na posicao [0][3] :)!

==> Digite a palavra para o caça-palavras ou 0, caso queira sair:
Matrizes
Palavra encontrada na posicao [2][2] :)!

==> Digite a palavra para o caça-palavras ou 0, caso queira sair:
aleatorio
Palavra não encontrada... :(

==> Digite a palavra para o caça-palavras ou 0, caso queira sair:
0
Program ended with exit code: 0
```

Caso o usuário digite uma palavra que não seja o comando de sair, seu programa deve responder se a palavra existe ou não na matriz. Além disso, caso ela exista, deve ser informada a posição da matriz onde a palavra se encontra, como no exemplo acima.

Regras muito importantes:

- Implementar um TAD Matriz, como discutimos em sala de aula. Ou seja, espero, ao menos, 3 arquivos na resposta (2 para o TAD e um para o cliente – main);
- Usar somente **alocação dinâmica** para strings e matrizes;
- Usar tipo opaco estruturado (estilo: `typedef struct x Tx`), como discutimos em sala de aula;
- Manipulação de arquivos para entrada somente na main (não manipular arquivos no TAD);
- Liberar toda a memória alocada! Use Valgrind...